

Altering Data In Server

- **React** -> **talks to** -> **json-server** -> **edits** -> **db.json**
- *axios* is middle man between **React** and **Server**;

Important note on hook useState

- why useState initialized with empty array [] or empty string '' ? what will happen if we init with blank state
- Millisecond 1: React instantly draws the screen.
- Millisecond 2: useEffect triggers the database download.
- Look at the bottom of your code. At Millisecond 1, React is forced to read this line of code:
notesToShow.map(note => ...);

If you left your state completely blank (useState()), **then notes would be undefined**. When JavaScript tries to run .map() on undefined, it completely panics. **It throws a massive red error: TypeError: Cannot read properties of undefined (reading 'map')** and your entire website crashes before the database even has a chance to respond!

By initializing it with an empty array [], you are giving React a safe placeholder. At Millisecond 1, React runs .map() on the empty array. **It says, "Okay, there are 0 things in this array, so I will draw 0 things on the screen."** It doesn't crash! It just waits peacefully until the data arrives a second later.

REST

- In REST terminology, we refer to individual data objects, such as the notes in our application, as resources.
- Every resource has a unique address associated with it - its URL.
- According to a general *convention used by json-server*, we would be able to locate an individual note at the resource URL *notes/3*, where *3 is the id* of the resource.
 - Resources are fetched from the server with HTTP GET requests.
 - For instance, an HTTP GET request to the URL notes/3 will return the note that has the id number 3.
 - An HTTP GET request to the notes URL would return a list of all notes.
- Creating a new resource for storing a note is done by making an HTTP POST request to the notes URL according to the REST convention that the json-server adheres to.
 - The data for the new note resource is sent in the body of the request
 - json-server requires all data to be sent in JSON format

- What this means in practice is that the data must be a correctly formatted string and that the request must contain the *Content-Type request header* with the *value application/json*.

Sending Data to the Server

- In previous part we successfully understand how to fetch data by using **React-hook Effect** or **useEffect** with `axios.get()` method.

```
useEffect(() => {
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      const { data } = response;
      setNotes(data)
    })
}, []);
```

- Now it's time to send the data to the server, we have already *fake json-server* with *db.json*, `axios.post()` method will help us to send the data to that server okay.

```
const addNote = (event) => {
  event.preventDefault();

  const newObject = {
    content: newNote,
    important: Math.random() < 0.5
  };

  // Adding Note to the server (fake-db) and to the local-machine via
  axios.POST() Method
  axios
    .post('http://localhost:3001/notes', newObject) // its take to param
    one is URL and second is response
    .then(response => {
      const { data } = response;
      setNotes([...notes, data])
      setNewNote('')
      console.log(response)
    })
};
```

Changing the Importance of Notes (manipulating data by **axios.put()**)

- `axios.put(location, changedData)`

```
const toggleImportanceOf = id => {
  const url = `http://localhost:3001/notes/${id}`
  const note = notes.find(n => n.id === id)
  const changedNote = { ...note, important: !note.important }

  axios
    .put(url, changedNote)
    .then(response => {
      setNotes(notes.map(note => note.id === id ? response.data : note))
      // response.data is changedNote here (new)
    })
}
```

- the first line defines the unique URL for each note resource based on its id.
- The array find method is used to find the note we want to modify, and we then assign it to the note variable.
- After this, we create a new object that is an exact copy of the old note, apart from the important property that has the value flipped (from true to false or from false to true)
- the `important` property gets the negation of its previous value in the original object
- After that its time to change in db so, we are invoking `axios.put()` method here to make change in `db.json`. `put(location, newData)` took two arguments:

```
setNotes(notes.map(note => note.id === id ? response.data : note))
```

- so here we are manipulating data into `main state` by map method by matching its id, `response.data` is acting behind the scenes is `newData`: `note` means old data;
- The map method creates a new array by mapping every item from the old array into an item in the new array. In our example, the new array is created conditionally so that **if `note.id === id` is true**; the note object returned by the server is added to the array. **If the condition is false, then we simply copy the item from the old array into the new array instead.**

Extracting Communication with the Backend into a Separate Module

- Right now, your `App.jsx` file is doing too many jobs. It is drawing HTML, it is managing React State, it is filtering arrays, and it is talking to the backend database using Axios. In the real world, this makes `App.jsx` massive and impossible to read.
- This section teaches you a massive architectural upgrade: **Separation of Concerns**.
- This file is what we call a Service (which is why we put it in a `services` folder). A Service is just a helper file that does the dirty work of talking to a database.

- The App component has become somewhat bloated after adding the code for communicating with the backend server. In the spirit of the single responsibility principle, we deem it wise to extract this communication into its own module.

Cleaner Syntax for Defining Object Literals

- old way of exporting

```
export default {  
  getAll: getAll,  
  create: create,  
  update: update  
}
```

- new way of exporting (ES6+ feature)

```
export default {getAll,create,update}
```

Promises and Errors

- We had previously mentioned that a promise can be in one of three different states. When an axios HTTP request fails, the associated promise is rejected. Our current code does not handle this rejection in any way.
- The rejection of a promise is handled by providing the then method with a second callback function, which is called in the situation where the promise is rejected.
- The more common way of adding a handler for rejected promises is to use the catch method.
- In practice, the error handler for rejected promises is defined like this:

```
axios  
  .get('http://example.com/probably_will_fail')  
  .then(response => {  
    console.log('success!')  
  })  
  .catch(error => {  
    console.log('fail')  
  })
```

- If the request fails, the event handler registered with the catch method gets called.

Exercises

- mess

```
import { useState, useEffect } from "react";
import personServices from './services/persons';

const Filter = ({ value, handler }) => {
  return (
    <>
      Filter shown with <input value={value} onChange={handler} />
    </>
  )
};

const PersonForm = ({ onSubmit, newName, userNameHandler, newNumber,
  userNumberHandler }) => {
  return (
    <form onSubmit={onSubmit} >

      <div>
        name: <input value={newName} onChange={userNameHandler} />
      </div>

      <div>
        number : <input value={newNumber} onChange={userNumberHandler} />
      </div>

      <div>
        <button type="submit">add</button>
      </div>

    </form>
  )
};

const Persons = ({ personsToShow, deleteHandler }) => {
  return (
    <div>
      {personsToShow.map(person => {
        const { id, name, number } = person;
        return (
          <p key={id}>{name} {number} <button onClick={() => deleteHandler(id,
name)}>delete</button></p>
        )
      })}
    </div>
  )
};

const App = () => {

  //
  const [persons, setPersons] = useState([]);
  // Locked html Inputs into React state here;
```

```
const [newName, setNewName] = useState('');
const [newNumber, setNewNumber] = useState('');
const [searchQuery, setSearchQuery] = useState('');

// fetching data here
useEffect(() => {
  personServices
    .getAll()
    .then(data => {
      setPersons(data)
    })
}, [])

const addPerson = (event) => {

  event.preventDefault();

  let isDuplicateName = persons.find((person) => person.name === newName);

  // Duplicate Checking
  if (isDuplicateName) {

    if (isDuplicateName.number !== newNumber) {

      let confirmation = window.confirm(`${newName} is already added to
      phonebook, replace the old number with a new one ?`);

      if (confirmation) {
        const { id } = isDuplicateName;
        const changedNumber = { ...isDuplicateName, number: newNumber };
        personServices
          .update(id, changedNumber)
          .then(updatedPerson => {
            setPersons(persons.map(person => person.id === id ? updatedPerson :
            person))
            setNewName('')
            setNewNumber('')
          })
      } else {
        setNewName('')
        setNewNumber('')
      }
    } else {
      alert(`${newName} is already added to phonebook`);
      setNewName('');
      setNewNumber('')
    }
    return
  };

  // object that contains user input
  const personObject = {
    name: newName,
    number: newNumber
```

```

    };

    // Connection to the backend (fake-server)
    personServices
      .create(personObject)
      .then(returnedPerson => {
        setPersons([...persons, returnedPerson])
        setNewName('')
        setNewNumber('')
      })
    };

    //
    const deletePersonOf = (id, name) => {
      if (window.confirm(`Delete ${name}`)) {
        personServices
          .remove(id)
          .then(() => (setPersons(persons.filter(person => person.id !== id))))
      }
    };

    // handler for name
    const handleUserName = (event) => {
      setNewName(event.target.value)
    };

    // handler for number
    const handleUserNumber = (event) => {
      setNewNumber(event.target.value)
    };

    // handler for searchBar
    const handleUserQuery = (event) => {
      setSearchQuery(event.target.value)
    };

    // Filtration on typed
    const personsToShow = persons.filter(person =>
    person.name.toLowerCase().includes(searchQuery.toLowerCase()));

    return (
      <div>
        <h1>Phonebook</h1>

        <Filter value={searchQuery} handler={handleUserQuery} />

        <h2>add a new</h2>

        <PersonForm onSubmit={addPerson} newName={newName} userNameHandler=
        {handleUserName} newNumber={newNumber} userNumberHandler={handleUserNumber} />

        <h2>Numbers</h2>
        <Persons personsToShow={personsToShow} deleteHandler={deletePersonOf} />
      </div>
    );
  }
}

```

```

    </div>
  )
};

export default App;

```

there is no replacement for if block but by returning we can skip else if condition is not so hard

- improvised

```

import { useState, useEffect } from "react";
import personServices from './services/persons';

const Filter = ({ value, handler }) => {
  return (
    <>
      Filter shown with <input value={value} onChange={handler} />
    </>
  )
};

const PersonForm = ({ onSubmit, newName, userNameHandler, newNumber,
  userNumberHandler }) => {
  return (
    <form onSubmit={onSubmit} >

      <div>
        name: <input value={newName} onChange={userNameHandler} />
      </div>

      <div>
        number : <input value={newNumber} onChange={userNumberHandler} />
      </div>

      <div>
        <button type="submit">add</button>
      </div>

    </form>
  )
};

const Persons = ({ personsToShow, deleteHandler }) => {
  return (
    <div>
      {personsToShow.map(person => {
        const { id, name, number } = person;
        return (
          <p key={id}>{name} {number} <button onClick={() => deleteHandler(id,
            name)}>delete</button></p>

```



```

    )
  })}
</div>
)
};

const App = () => {

  // ==== useState ==== //
  const [persons, setPersons] = useState([]);
  // name
  const [newName, setNewName] = useState('');
  // number
  const [newNumber, setNewNumber] = useState('');
  // searchInput
  const [searchQuery, setSearchQuery] = useState('');

  // ==== fetching data here ==== //
  useEffect(() => {
    personServices
      .getAll()
      .then(data => {
        setPersons(data)
      })
  }, [])

  // ==== Functions ==== //
  const addPerson = (event) => {
    event.preventDefault();

    // Finding Entry here
    let isDuplicate = persons.find((person) => person.name === newName);

    // Checking for duplicate
    if (isDuplicate && isDuplicate.number === newNumber) {
      alert(`${newName} is already added to phonebook`);
      setNewName('');
      setNewNumber('');
      return;
    };

    // Cheking for non duplicate
    if (isDuplicate && isDuplicate.number !== newNumber) {
      let confirmation = window.confirm(`${newName} is already added to phonebook, replace the old number with a new one ?`);

      if (confirmation) {
        let changedNumber = { ...isDuplicate, number: newNumber };
        personServices
          .update(isDuplicate.id, changedNumber)
          .then(updatedPerson => {
            setPersons(persons.map(p => p.id === isDuplicate.id ? updatedPerson :
p))
            setNewName('')

```

```
        setNewNumber('')
      })
    }
    else {
      setNewName('')
      setNewNumber('')
    }
    return
  }

  // object that contains user input
  const personObject = {
    name: newName,
    number: newNumber
  };

  // Connection to the backend (fake-server)
  personServices
    .create(personObject)
    .then(returnedPerson => {
      setPersons([...persons, returnedPerson])
      setNewName('')
      setNewNumber('')
    })
  };

  //
  const deletePersonOf = (id, name) => {
    if (window.confirm(`Delete ${name}`)) {
      personServices
        .remove(id)
        .then(() => (setPersons(persons.filter(person => person.id !== id))))
    }
  };

  // Filtration on typed
  const personsToShow = persons.filter(person =>
person.name.toLowerCase().includes(searchQuery.toLowerCase()));

  // ==== Handlers ==== //
  // handler for name
  const handleUserName = (event) => {
    setNewName(event.target.value)
  };

  // handler for number
  const handleUserNumber = (event) => {
    setNewNumber(event.target.value)
  };

  // handler for serachBar
  const handleUserQuery = (event) => {
    setSearchQuery(event.target.value)
```

```

};

return (
  <div>
    <h1>Phonebook</h1>

    <Filter value={searchQuery} handler={handleUserQuery} />

    <h2>add a new</h2>

    <PersonForm onSubmit={addPerson} newName={newName} userNameHandler=
{handleUserName} newNumber={newNumber} userNumberHandler={handleUserNumber} />

    <h2>Numbers</h2>
    <Persons personsToShow={personsToShow} deleteHandler={deletePersonOf} />
  </div>
)
};

export default App;

```

Mentor session and Verdict of this chapter

- Question 1: Explain the difference between POST, PUT, and DELETE in REST:
 - Which ones need a data object (body) sent with them, and which one only needs a URL with an id? What is the specific job of each method on the backend?

q.1 =>

- post and put both needed body , post save new entry from newly typed userinput , put just needed updated body with id (location) , delete needs id only
 - newly we used hardcoded data okay but in real case cant save this hardcoded data for while beacuse we need modify them and reder to internet back , so should be saved somewhere on internet okay , course introduced me backend fake-server okay so puted that data into that to acces that we have method get , post , put , delete there are four main operations okay get we getting fetching data from internet where we saved post we are taking data from browser react app and then send to server put updating data on buisness logic delete we destroying that entry from db or internet permmently
- Mentor hitted me with this
 1. Question 1: HTTP Methods (100% - PERFECT)
 2. Your answer here was rock-solid:
 - **POST:** Sends a new body to create an entry.
 - **PUT:** Sends an updated body + targeting ID to replace/update an entry.

- **DELETE:** Needs only the target ID URL to destroy an entry.
- **GET:** Reads/fetches data.
- You completely understand REST CRUD. Flawless.

- Question 2: React Immutability & Array Methods

- When syncing our React state after a server update:
 - Why do we use `.map()` with a ternary operator (? 😊 when updating an item (PUT)?
 - Why do we use `.filter()` with `!==` when removing an item (DELETE)?
 - Why are we forbidden from just doing `persons.splice()` or `persons[i] = newPerson?`

q.2 =>

- well my basics are clear about arrays but this question seems to me new why we do use this methods
- well as i know its new es6+ feature that handle traditional chores of mapping data and replace if else ladder ,
- well again this are all high ordered functions automate lots of things i mean save lot line of code we can use vanilla method to filter out things
- same answer i mean if js giving us power of mutating data very easy way why we would take long way ??
- Mentor hit me with this

1. Question 2: Immutability & Array Methods (MISSED - CRITICAL REACT CONCEPT)

2. You answered that `.map()` and `.filter()` are just shorter ES6 syntax. While they are cleaner, *that is NOT the reason* **React forces us to use them.**

3. Here is the golden rule of **React: NEVER MUTATE STATE DIRECTLY.**

4. Why can't we use `persons.splice()` or `persons[0] = newObj?`

Methods like `.splice()`, `.push()`, or direct assignment `persons[0] = ...` mutate (modify) the existing array in-place in memory.

React checks if state changed by looking **at the memory reference of the array.** *If you modify the same array in place, React thinks: "The memory address is the same, nothing changed!" and it will NOT re-render the screen.*

5. Why do we use `.map()` and `.filter()`?

Both `.map()` and `.filter()` **never touch the old array. They create and return a BRAND NEW array in memory.** new Array means new memory reference and this reference react gets from `useState` , hook state is `useState` for react and react re-renders the screen instantly!

6. **Rule to remember:** In React, `.map()` and `.filter()` are used because **they are pure / immutable** — they *return fresh copies instead of mutating existing state*.

- Question 3: Services & Separation of Concerns

Why do we extract our network requests into a separate file (`services/persons.js`) and return `response.data` instead of writing raw `axios.get()` or `axios.post()` directly inside `App.jsx`? What problem does this solve?

q.3 =>

- mostly App component will get massive time by time and We are if overrulling DRY so just defined once and accese whener we wanted and it will prevents from clutter may be noy sure correct me here if wrong

- Mentor hitted me with this

0. Question 3: Services & Separation of Concerns (85% - VERY GOOD)

1. You nailed the main points: preventing `App.jsx` from becoming massive, reducing clutter, and following DRY (Don't Repeat Yourself).

2. The extra bonus point: **It gives us Separation of Concerns.**

- **`App.jsx` should only care about UI & User Interaction** (*buttons, forms, state*).
 - **`services/persons.js` should only care about HTTP & Network Protocols** (*Axios, URLs, formatting response.data*)
 - If your backend URL changes tomorrow from `localhost:3001` to `api.myphonebook.com`, you only have to edit one line in `persons.js`, and `App.jsx` never has to change at all.
-

- Question 4: Promises and Errors (`.catch`)

Imagine you try to update a person's phone number using `axios.put()`, but someone else deleted that person from the database 5 seconds ago (the server responds with a 404 Not Found error):

- What happens to your `.then()` block?
- What block runs instead, and what two things should you do inside of it to clean up the UI?

q.4 =>

- not sure
- handle the erroe by `.error ()` blockGame belowe above

- Mentor hitted me with this

1. Question 4: Promises and Errors (MISSED - NEEDS REINFORCEMENT)

2. Let's review what happens when an API call fails (like a 404 Not Found):

3. What happens to `.then()`?

* It is completely skipped. **JavaScript will not run a single line inside `.then()`**.

2. What runs instead?

* The `.catch(error => { ... })` block (remember the method name is `.catch()`, not `.error()`).

3. What 2 things should you do inside `.catch()`?

* **Notify the user:** `alert("This person was already removed from the server")`

* **Clean the UI:** **Remove the ghost person from React state** so the UI matches reality: `setPersons(persons.filter(p => p.id !== id))` why ? *db changes* nothing to do with `reactState` we have to take care of it